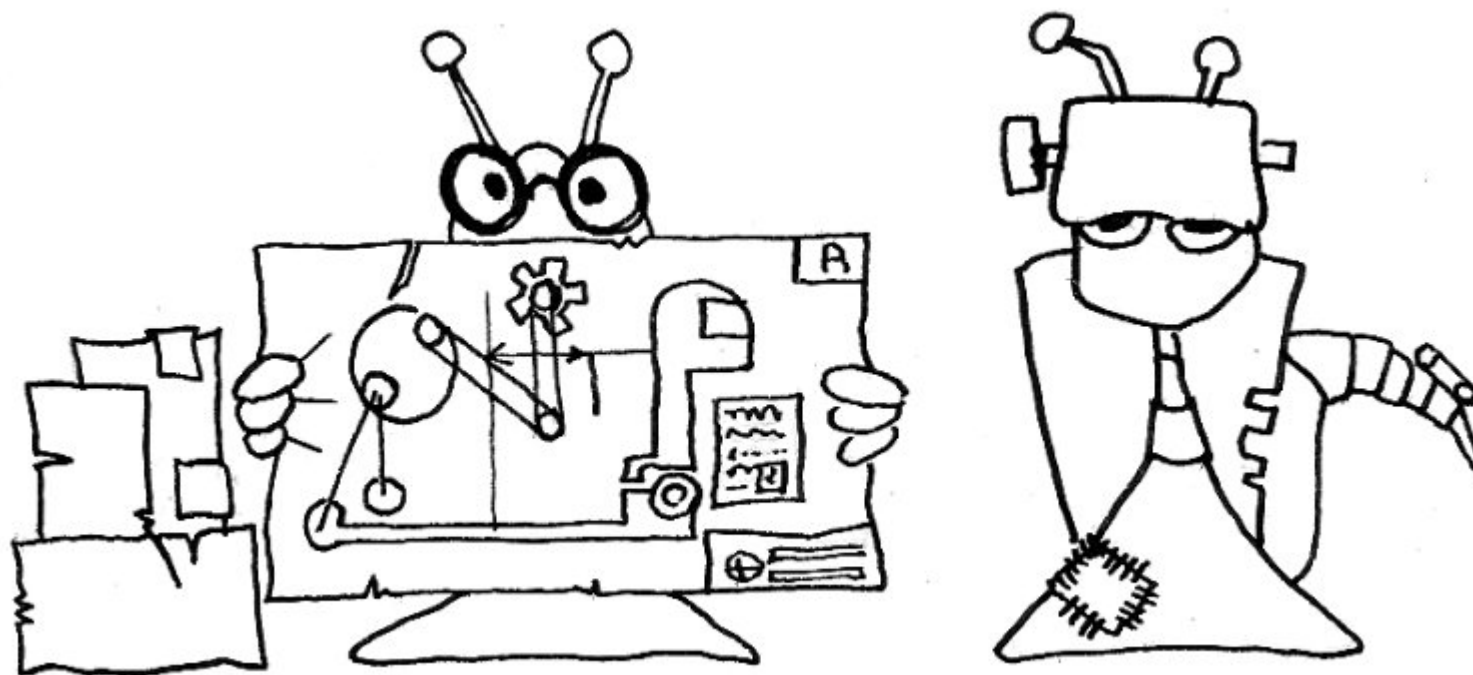


Chapitre 3: Services, Interfaces et Appels Système



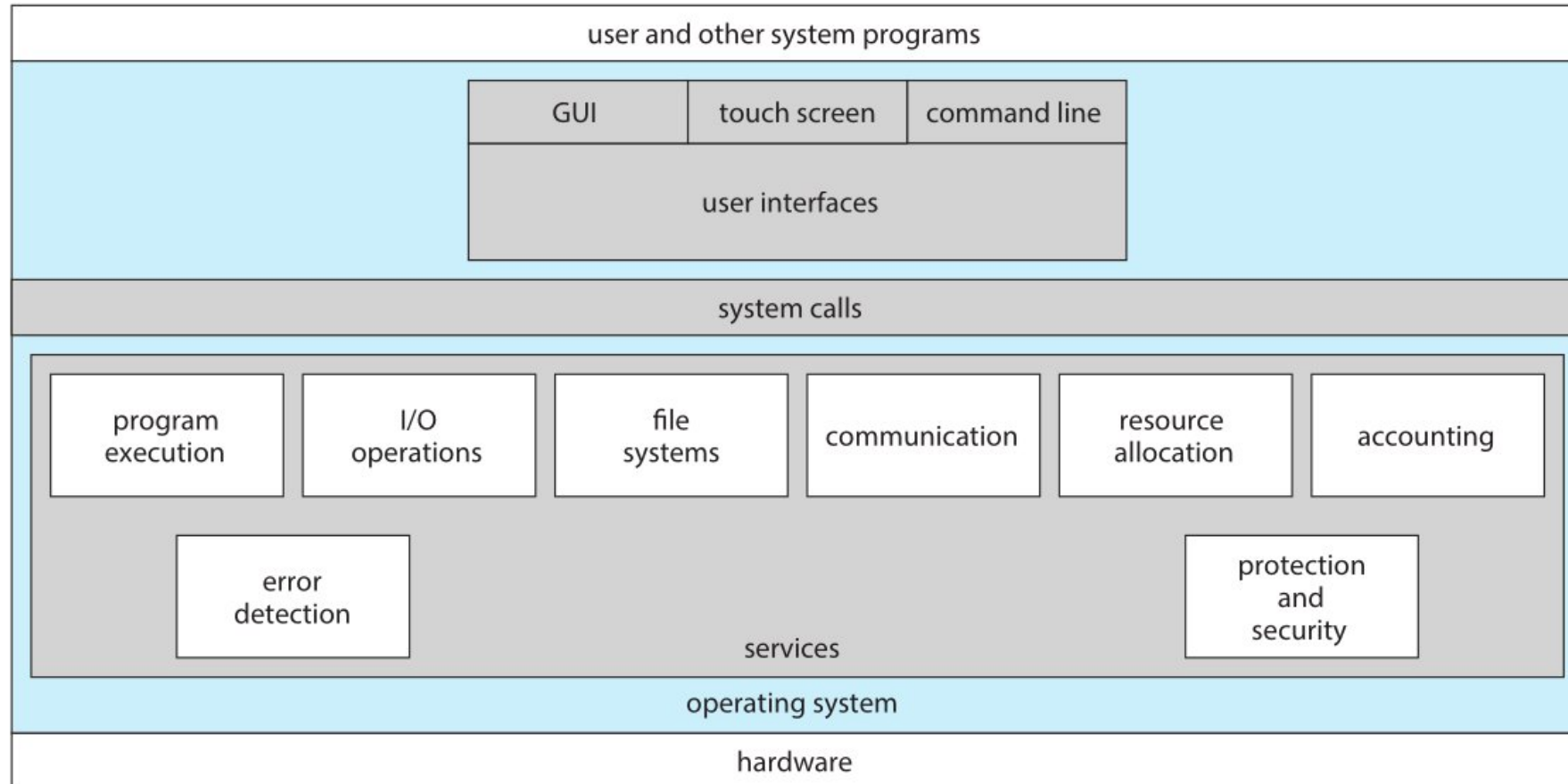
Objectifs du chapitre

1. Lister les catégories de services que le SE offre aux utilisateurs et au système lui-même
2. Expliquer le mécanisme d'un appel système : libc, trap, changement de mode, retour
3. Distinguer les six catégories d'appels système POSIX avec un exemple concret
4. Décrire le rôle de l'éditeur de liens et du chargeur
5. Tracer un programme C avec **strace** et identifier ses appels système
6. Comparer les structures monolithique, micro-noyau, modulaire et hybride

Section 1 : Les services du système d'exploitation

Vue d'ensemble → services pour l'utilisateur → services pour l'efficacité → exemple du serveur web

Vue d'ensemble des services du SE



A view of operating system services.

Les services pour l'utilisateur

- **Exécution de programmes** : charger un programme en mémoire et l'exécuter, terminer normalement ou anormalement
- **Opérations d'E/S** : lire et écrire des fichiers et des périphériques ; les utilisateurs ne contrôlent pas directement les périphériques
- **Manipulation du système de fichiers** : créer, lire, écrire, supprimer, rechercher, lister, gérer les permissions
- **Communication** : échange d'informations entre processus (mémoire partagée ou passage de messages)
- **Détection d'erreurs** : surveiller CPU, mémoire, périphériques et programmes ; réagir de façon adaptée

Les services pour l'efficacité du système

- **Allocation de ressources** : gérer la concurrence entre processus — CPU (ordonnancement), mémoire, périphériques
- **Comptabilité (accounting)** : enregistrer quels programmes consomment quelles ressources, pour la facturation ou les statistiques
- **Protection et sécurité** : contrôler l'accès aux ressources ; authentifier les utilisateurs ; défendre les périphériques réseau
- Ces services existent pour le système lui-même, pas directement pour l'utilisateur

Chaîne de sécurité : un système n'est aussi fort que son maillon le plus faible

Exemple : un serveur web et les services du SE

- Un serveur web (nginx, Apache) est un programme comme un autre : il a besoin du SE pour presque tout
- **Communication** : ouvre une socket TCP (port 80) et accepte les connexions réseau
- **Fichiers** : lit les pages HTML sur le disque via le système de fichiers
- **E/S** : envoie les réponses sur la carte réseau et le réseau
- **Exécution de programmes** : crée un processus (ou un thread) par connexion concurrente
- **Détection d'erreurs** : logge les requêtes invalides, gère les disques pleins

Sans les services du SE, chaque serveur devrait réimplémenter le réseau et les fichiers

Section 2 : Les interfaces utilisateur et SE

CLI → GUI → tactile → interpréteur de commandes → CLI vs GUI : mêmes appels système

CLI, GUI, tactile : trois interfaces

- **Interface en ligne de commande (CLI)** : l'utilisateur tape des commandes textuelles avec options et arguments
- **Interface graphique (GUI)** : fenêtres, menus et icônes pilotés à la souris
- **Interface tactile** : gestes du doigt sur l'écran, clavier virtuel
- Certains systèmes offrent les trois variantes (Windows, macOS, Android)

CLI

GUI

Le choix de l'interface est une affaire de préférence personnelle et de contexte d'usage

L'interpréteur de commandes (shell)

- **Interpréteur de commandes** : programme spécial qui lit et exécute les commandes de l'utilisateur
- Sur les systèmes Unix/Linux, on parle de **shell** (bash, sh, csh, ksh, zsh)
- Deux modes d'implémentation :
 - le shell contient lui-même le code de la commande (commandes internes)
 - le shell lance un programme externe (système Unix : **rm**, **cp**, ...)
- **Le shell ne comprend pas les commandes** : il charge le programme identifié par le nom

Ajouter une nouvelle commande = ajouter un nouveau fichier exécutable

CLI vs GUI : les mêmes appels système

- Deux chemins, une même destination : **le noyau**
- **GUI** : cliquer sur «copier-coller» déclenche une suite d'appels système d'écriture
- **CLI** : taper `cp in.txt out.txt` déclenche la même suite d'appels système
- La CLI est plus efficace pour l'administrateur : scriptable, automatisable
- Sur certains systèmes, une partie des fonctions n'est disponible qu'en CLI

le GUI est un programme utilisateur de plus, qui sollicite le noyau comme tout le monde

Section 3 : Les appels système

Définition → API et libc → exemple (read) → mécanisme
→ numéros → paramètres → cas x86-64 → erreurs

Définition : l'appel système

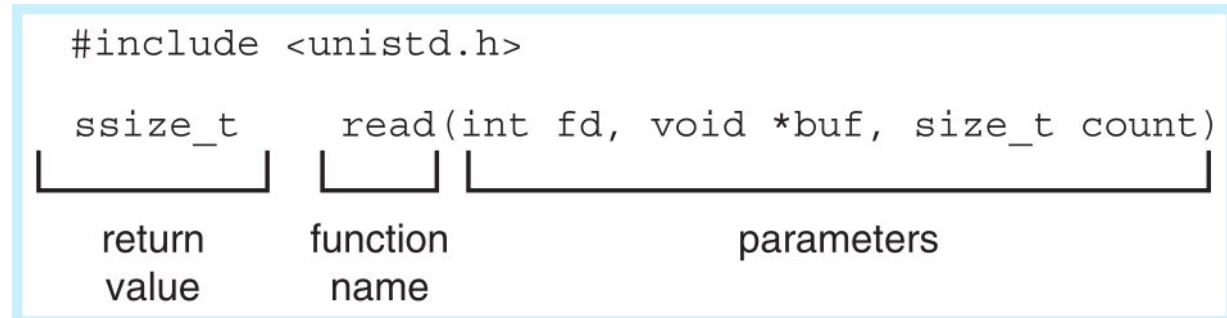
- **Appel système** : requête programmatique, émise depuis le mode utilisateur, demandant au noyau d'exécuter un service privilégié ; elle bascule le processeur en mode noyau.
- Les appels sont disponibles comme des fonctions C (et parfois en assembleur)
- **Exemple** : créer un fichier sur disque exige un appel système
- Le noyau vérifie la légitimité de la demande (permissions, accès)
- Un appel système interrompt le programme appelant, qui reprend à la fin
- Il est exécuté en mode noyau, même si son demandeur tourne en mode utilisateur

L'API et la bibliothèque C (libc)

- **API (Application Programming Interface)** : ensemble de fonctions disponibles au programmeur, avec leurs paramètres et valeurs de retour
- **Trois API majeures** : Windows API, POSIX API (Unix, Linux, macOS), Java API
- Sur Unix/Linux en C, la bibliothèque est **libc (man 3)**
- Les fonctions de l'API invoquent les appels système « en coulisses »
- Exemple : **CreateProcess()** (Windows) invoque **NTCreateProcess()** dans le noyau
- **Avantages de l'API** : portabilité du code, simplicité par rapport aux syscalls bruts

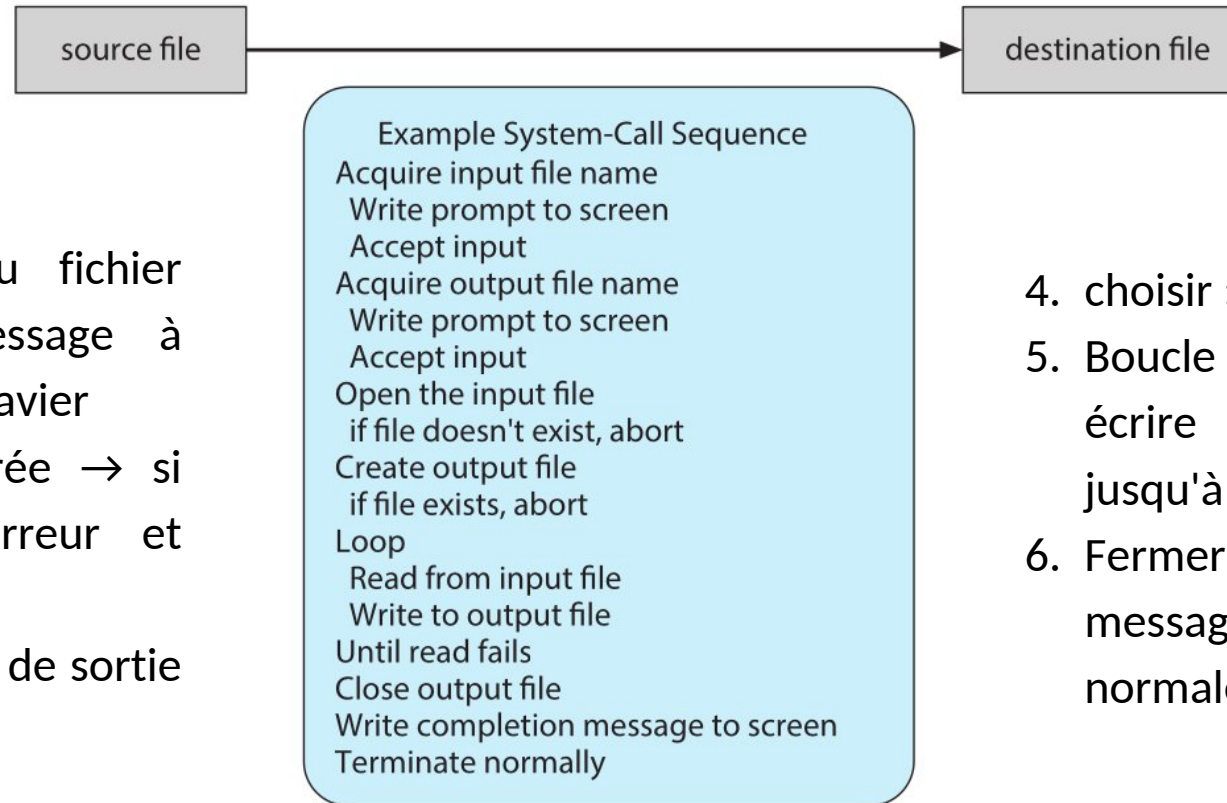
Exemple : la signature de read() (man 2)

- La page de manuel donne l'API : **man 2 read** (section 2 = appels système)
- **Signature** : `ssize_t read(int fd, void *buf, size_t count);`
- **fd** : descripteur du fichier à lire



- **buf** : tampon mémoire où placer les données
- **count** : nombre maximal d'octets à lire
- **Retour** : nombre d'octets lus ; 0 = fin de fichier ; -1 = erreur

Exemple : copier un fichier, séquence d'appels système



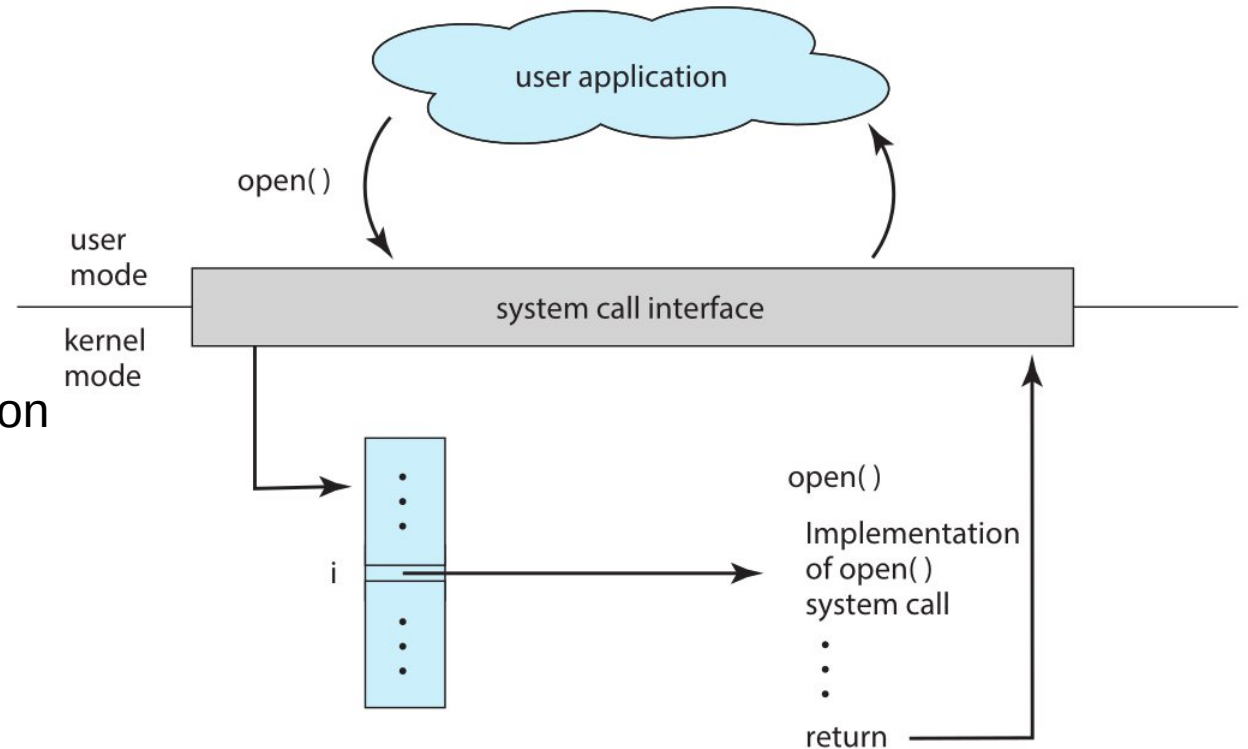
1. Demander le nom du fichier source, écrire le message à l'écran, lire la réponse clavier
2. Ouvrir le fichier d'entrée → si inexistant, afficher l'erreur et terminer
3. Créer et ouvrir le fichier de sortie → s'il existe déjà,

4. choisir : écraser ou abandonner
5. Boucle : lire dans le fichier d'entrée, écrire dans le fichier de sortie, jusqu'à la fin
6. Fermer les deux fichiers, afficher le message de fin, terminer normalement

Example of how system calls are used
cp in.txt out.txt

Mécanisme : de l'API au noyau

- En mode utilisateur : le programme appelle **open()** — la fonction de la **libc**
- **L'interface d'appels système** intercepte l'appel et invoque le syscall correspondant
- En mode noyau : exécution de l'implémentation d'**open()** par le SE
- Retour du statut de l'appel à l'appelant
- L'appelant n'a pas besoin de connaître l'implémentation : seule l'API compte
- La plupart des détails sont cachés par l'API et l'environnement d'exécution



The handling of a user application invoking the open() system call.

Numéros d'appels système et table de dispatch

- Chaque appel système possède un numéro unique
- L'interface d'appels système maintient une table indexée par ces numéros
- Exemples Linux x86-64 : `read` = 0, `write` = 1, `openat` = 257, `fork` = 57, `execve` = 59
- Le numéro est passé dans le registre `rax` sur x86-64
- Le noyau lit `rax` et saute vers l'implémentation correspondante

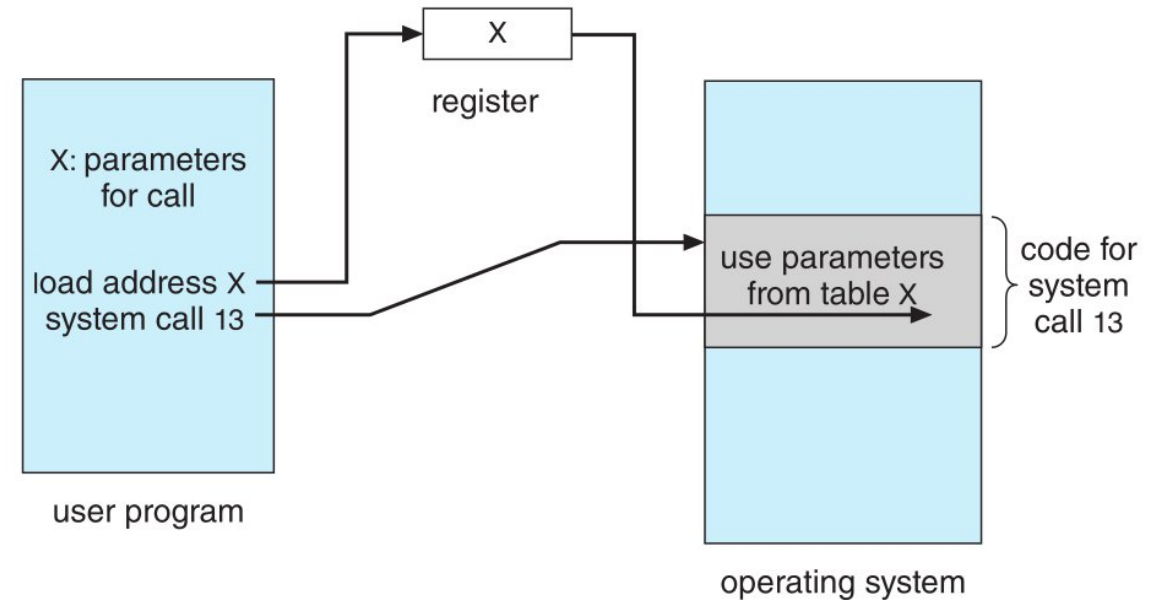
Conséquence : ajouter un syscall = l'ajouter au noyau et lui attribuer un `numéro`

Passage des paramètres au noyau

Trois méthodes possibles :

- **Registres** : la plus simple, si le nombre de paramètres est faible
- **Bloc en mémoire** : les paramètres sont stockés dans une table, on passe son adresse
- **Pile** : les paramètres sont empilés par le programme et dépilés par le SE

Linux combine : ≤ 5 paramètres $\rightarrow$ registres ; plus $\rightarrow$ bloc mémoire



Passing of parameters as a table

Avantage des méthodes bloc/pile :
pas de limite de nombre ni de longueur des paramètres

Section 4 : Les types d'appels système

Contrôle des processus → gestion des fichiers → périphériques →
maintenance → communication → protection

Les six catégories d'appels système

- **Contrôle des processus** : créer, terminer, charger, attendre, attribuer
- **Gestion des fichiers** : créer, supprimer, ouvrir, fermer, lire, écrire, repositionner
- **Gestion des périphériques** : demander, libérer, lire, écrire, attacher/détacher
- **Maintenance des informations** : date/heure, données système, attributs de processus et fichiers
- **Communication** : créer des connexions, envoyer/recevoir des messages, attacher des périphériques distants
- **Protection** : obtenir et modifier les permissions

Contrôle des processus : **fork**, **exec**, **exit**, **wait**

- **fork()** : crée un processus fils en dupliquant le processus courant
 - **execve()** : remplace l'image mémoire du processus par un nouveau programme
 - **exit()** : termine le processus et renvoie un code à son parent
 - **wait()** / **waitpid()** : le parent attend la fin d'un fils et récupère son statut
 - **getpid()** / **getppid()** : attributs du processus
-
- POSIX standardise cette famille ; la section 2 du manuel la documente

Gestion des fichiers : **open**, **read**, **write**, **close**

- **open(path, flags, mode)** : ouvre un fichier et retourne un descripteur
- **read(fd, buf, count)** : lire le descripteur du fichier
- **write(fd, buf, count)** : écrire dans un descripteur de fichier
- **lseek(fd, offset, whence)** : repositionner le pointeur de lecture/écriture
- **close(fd)** : libérer le descripteur
- **stat()** : attributs du fichier (taille, permissions, dates)
- POSIX standardise cette famille ; la section 2 du manuel la documente

Gestion des périphériques et maintenance des informations

- **Périphériques** : `read`, `write`, `ioctl` (contrôle de bas niveau)
- Sous Unix, un fichier spécial (`/dev/*`) représente chaque périphérique : mêmes syscalls que les fichiers
- **Maintenance** : `getpid()`, `getppid()`, `time()`, `gettimeofday()`, `get/setdate`
- **Données système** : `uname()` (nom, version, machine)
- **Lecture des attributs** : `stat()`, `getuid()`, `getgid()`
- Le noyau tient ces informations dans ses structures de données

Communication et protection

- **Communication** : échanges entre processus, locaux ou distants
- Deux modèles : **mémoire partagée** (lecture/écriture d'une zone commune) et **passage de messages** (le SE déplace les paquets)
- Appels POSIX : `pipe()`, `shmget()` (mémoire partagée), `socket()`, `send()`, `recv()` (réseau)
- Protection : `chmod()` et `chown()` modifient les permissions et le propriétaire
- Le chapitre 7 détaillera les permissions ; le réseau dépasse ce cours
- Ces appels illustrent la séparation : le SE transporte les messages sans les interpréter

Section 5 : Les programmes et services système

Catégories de programmes système → services et daemons → exemple : ls et sshd

Programmes système : sept catégories

1. **Gestion des fichiers** : créer, supprimer, copier, renommer, imprimer, lister (`mkdir`, `cp`, `rm`, `ls`)
2. **Informations d'état** : date, heure, mémoire disponible, espace disque, nombre d'utilisateurs (`date`, `cal`, `df`, `free`)
3. **Modification de fichiers** : éditeurs de texte, recherche et transformation (`nano`, `grep`, `sed`)
4. **Support de langages de programmation** : compilateurs, assembleurs, débogueurs (`gcc`, `gdb`)
5. **Chargement et exécution** : chargeurs absolus et relocalisables, éditeurs de liens
6. **Communication** : messagerie, navigation web, courriel, transfert de fichiers, connexion à distance (`ssh`, ...)
7. **Services d'arrière-plan**: Tous les systèmes à usage général disposent de mécanismes permettant de lancer certains processus de programmes système au démarrage (`subsystems`, ou `daemons`).

Les services système : les daemons

- **Daemons** : processus lancés au démarrage, exécutés en permanence ou selon un plan
- Services système, sous-systèmes ou daemons — toujours en cours d'exécution
- **Exemples** : serveur réseau qui écoute les connexions, planificateur de tâches, moniteur d'erreurs, serveur d'impression
- Un système typique en compte des dizaines
- Certains services s'exécutent en espace utilisateur via des daemons

Commande pour lister les daemons: `service --status-all`

Exemple : **ls** et **sshd**, des programmes système

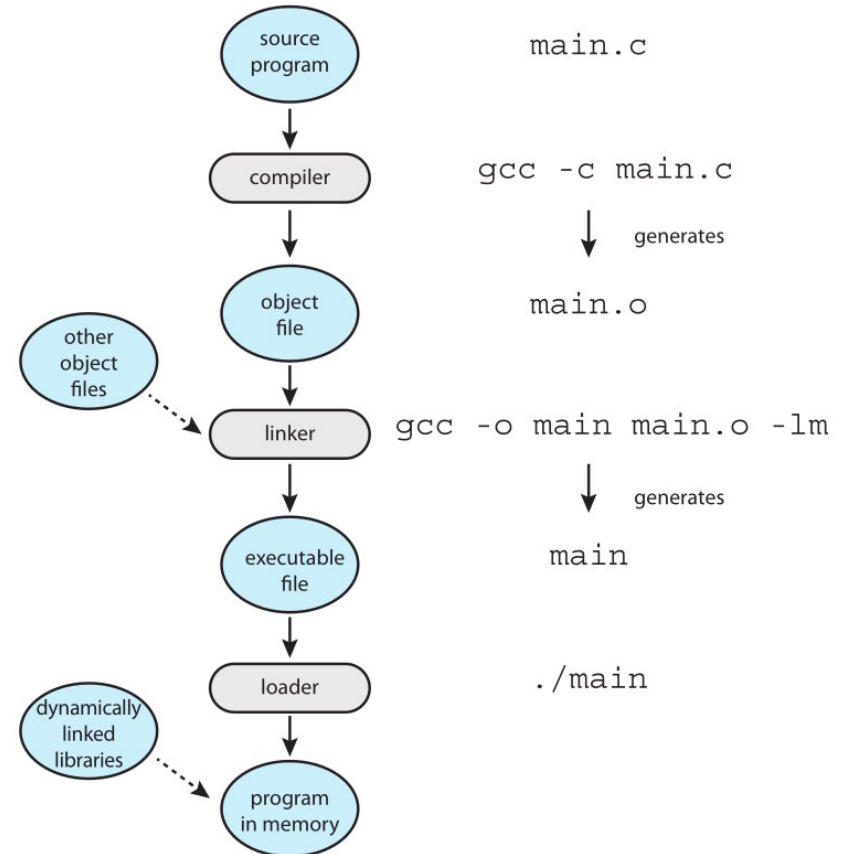
- **ls** : programme système de gestion de fichiers
- Appels système utilisés : **getdents()** (lister les entrées du répertoire) et **stat()** (attributs des fichiers)
- Chaque nom affiché par **ls -l** provient de ces appels système
- **sshd** : daemon de connexion à distance (SSH)
- Appels système : **socket()**, **bind()**, **listen()**, **accept()**
- Un programme système = une fenêtre sur les appels système du noyau

Section 6 : Compilation, édition de liens et chargement

De la source au processus → éditeur de liens → statique vs dynamique → ELF → ldd et ltrace

De la source au processus : étapes (1,2,3)

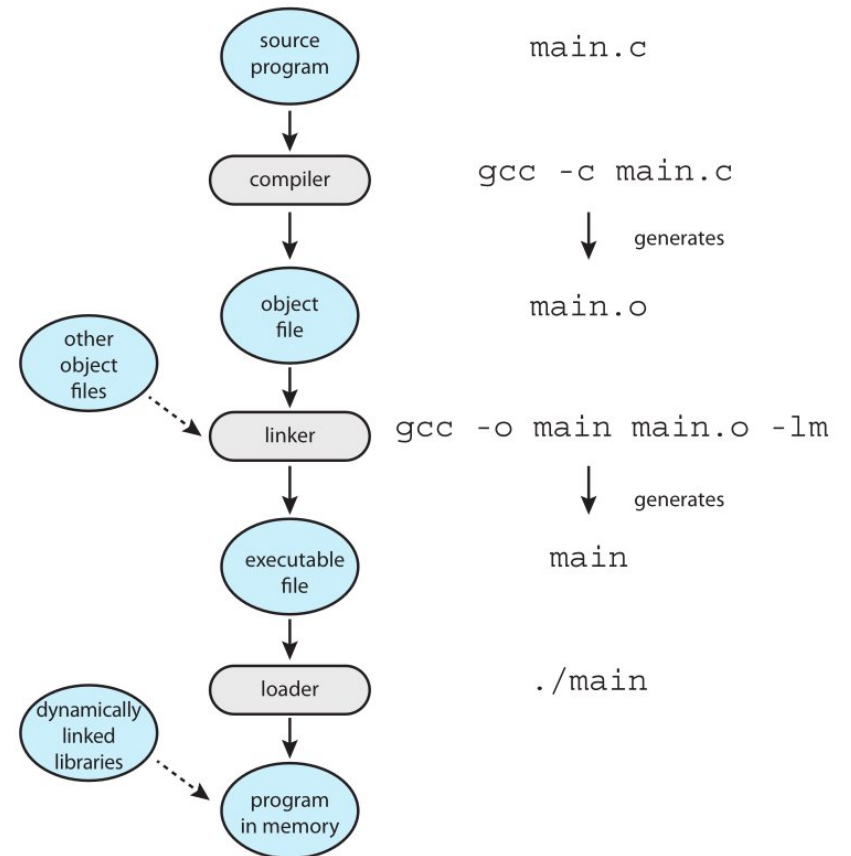
1. **Compilation** (**compiler**) : le compilateur transforme le code source en fichier objet relocalisable (`main.c` → `main.o`)
2. **Édition de liens** (**linker**) : l'éditeur de liens combine les fichiers objets et les bibliothèques en un exécutable (`main.o` + libs → `main`)
3. **Chargement** (**loader**) : le chargeur place l'exécutable en mémoire, dans le contexte d'un processus



The role of the linker and loader.

De la source au processus : étapes (4,5,6)

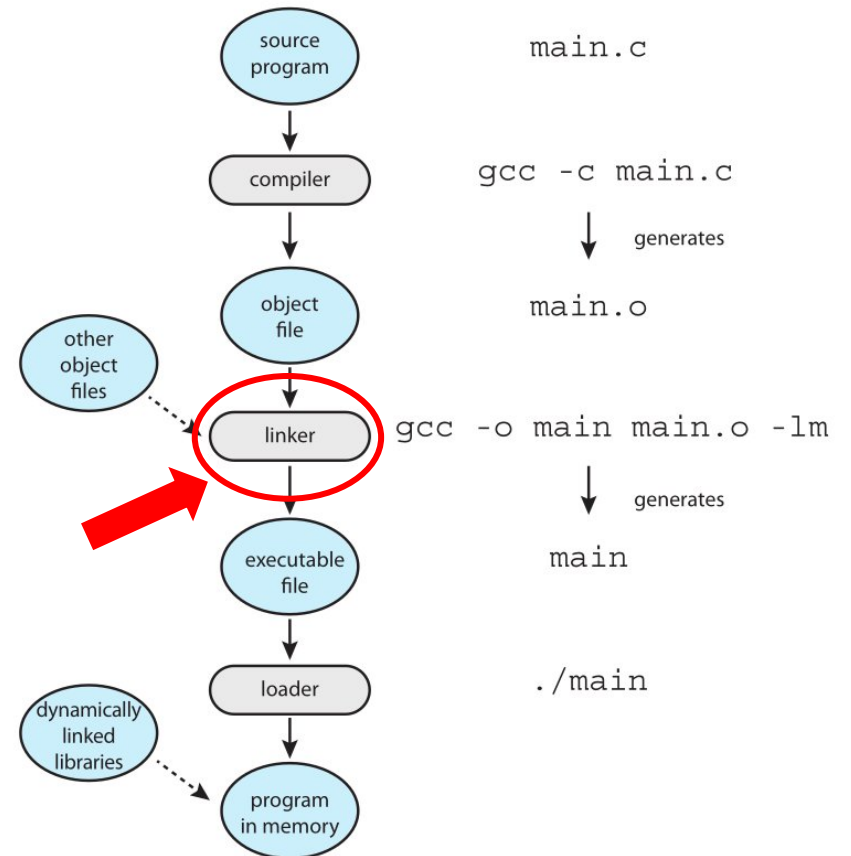
4. **Exécution** : le programme devient éligible pour un cœur de CPU
5. **Le shell déclenche le chargement** : `fork()` puis `execve()` quand vous tapez `./main`
6. **Relocalisation** : assigne les adresses finales et ajuste le code pour qu'il puisse appeler les fonctions des bibliothèques



The role of the linker and loader.

Le rôle de l'éditeur de liens (linker)

- **Entrées** : fichiers objets (main.o, fonction.o) + bibliothèques (stdio, stdlib, math)
- **Sortie** : l'exécutable (a.out)
- L'éditeur résout les **références** : chaque appel à printf() doit pointer vers son code
- Les fichiers crt0.o, crtbegin.o, crtend.o fournissent le démarrage : c'est eux qui appellent main()
- Les bibliothèques vivent dans /usr/lib et /lib
- Ajouter #include <stdio.h> ne suffit pas : le code de printf() vient des bibliothèques



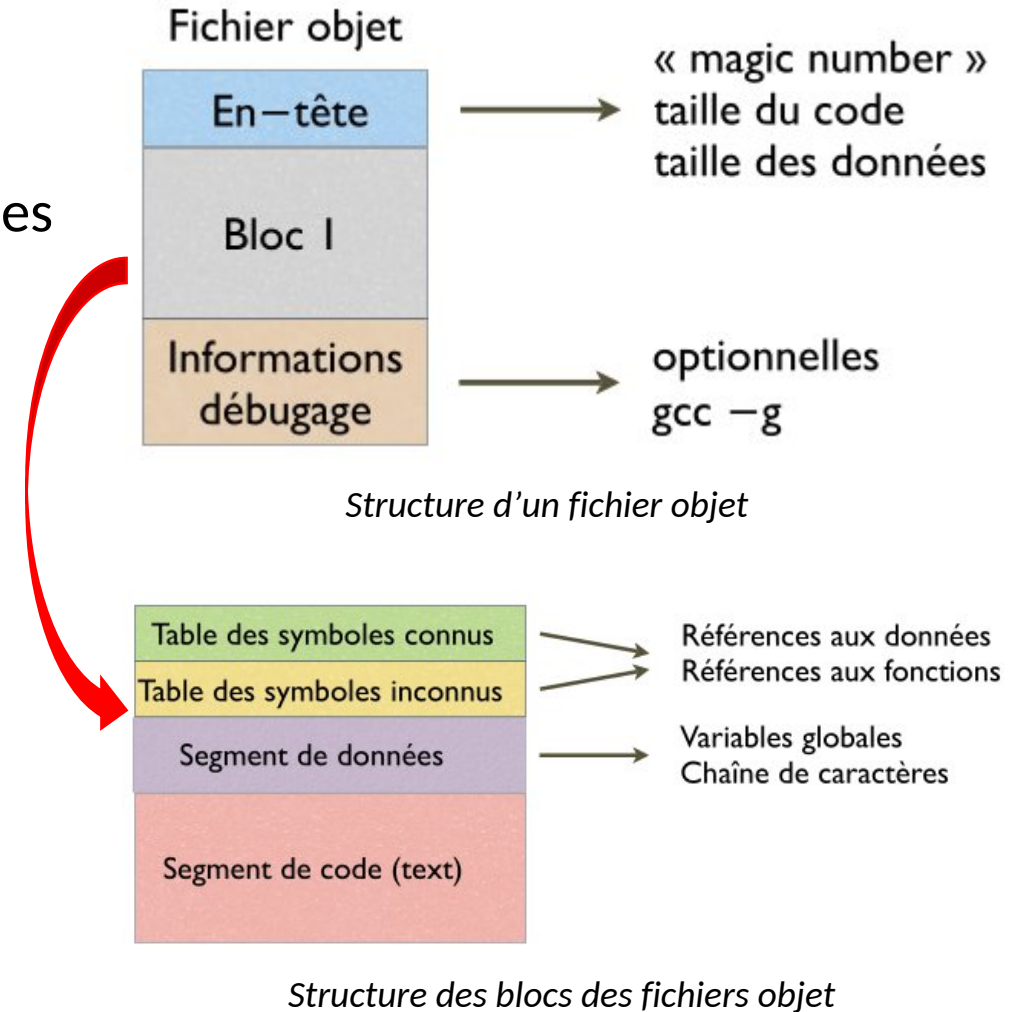
The role of the linker and loader.

Liaison statique vs dynamique

- **Liaison statique** : les bibliothèques sont copiées dans l'exécutable à la compilation → exécutable volumineux, autonome
- **Liaison dynamique** : le code des bibliothèques est chargé au moment du lancement par `ld.so` (*chargeur et éditeur de liens dynamiques par défaut sous Linux*)
- **Avantages du dynamique** : exécutables plus petits, mise à jour des bibliothèques sans recompilation, partage mémoire entre processus
- **Inconvénient** : dépendance aux bibliothèques présentes sur la machine cible
- **Exemple** : `gcc -static main.c` vs `gcc main.c` → comparer avec `ls -l`
- Sur Linux, les bibliothèques partagées portent l'extension `.so`

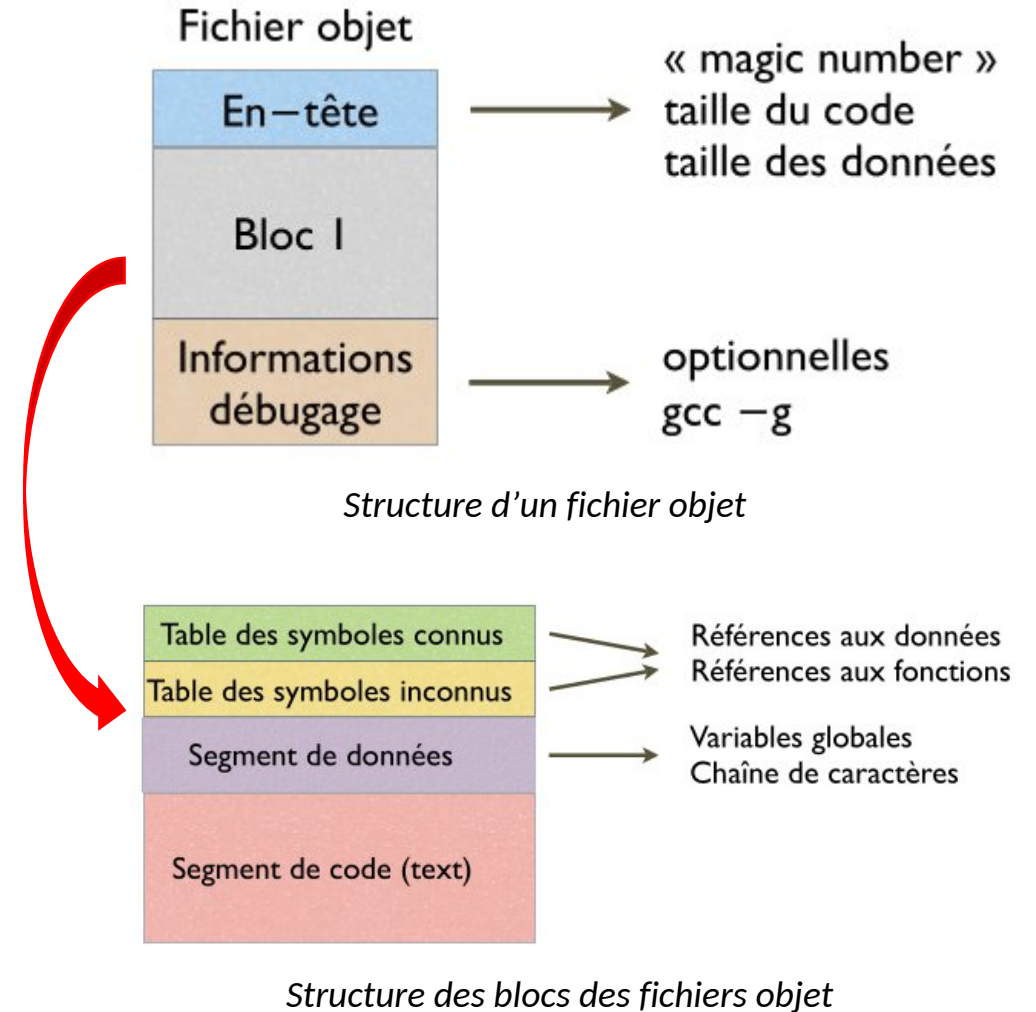
ELF : structure des fichiers exécutables

- **ELF (Executable and Linkable Format)** : format standard des exécutables, objets et bibliothèques sous Linux
- **Structure** : en-tête (magic number, tailles) + sections + optionnellement infos de débogage
- **Sections principales** : **table des symboles** (connus/inconnus), **table des chaînes**, **segment texte** (code), **segment de données**



ELF : structure des fichiers exécutables

- Un fichier objet, une bibliothèque et un exécutable ont la même forme
- **Outils** : `file` (type du fichier), `readelf` (analyse), `size` (taille des segments)
- L'adresse d'entrée (entry point) désigne la première instruction exécutée



Exemple : ldd et file

- `ldd /bin/ls` : liste les bibliothèques partagées dont dépend un exécutable
- `file main.o` : reconnaît le type (ELF relocisable, exécutable, bibliothèque)
- `nm -u main` : symboles indéfinis — les fonctions à résoudre au chargement
- **Exemple:** `ldd /bin/su` révèle **libpam, libc, libcrypt, ...**

Vérifier les dépendances d'un programme avant de le déployer :

RÉFLEXE D'ADMINISTRATEUR SYSTEME

Section 7 : Pourquoi les applications sont spécifiques au SE?

Obstacles à la portabilité → trois voies → API vs ABI → WSL et Wine

Les obstacles à la portabilité

- **Format binaire** : chaque OS impose un format d'exécutable (ELF sous Linux, PE sous Windows, Mach-O sous macOS)
- **Jeu d'instructions** : les processeurs diffèrent (x86 vs ARM) ; les instructions doivent être compatibles
- **Appels système** : chaque OS a ses propres appels — numéros, paramètres, significations, retours différents
- **Bibliothèques et API** : les bibliothèques du système fournissent des services spécifiques (GUI, etc.)

Résultat : un binaire compilé sur un OS ne s'exécute pas sur un autre

Ces différences expliquent le travail colossal de portage d'un navigateur comme Firefox

Trois voies pour la portabilité

- **Langage interprété** : le code source est exécuté par un interpréteur disponible sur chaque OS (Python, Ruby) — coût : performance, fonctionnalités limitées
- **Machine virtuelle** : le programme s'exécute dans un environnement d'exécution porté partout (Java/JVM) — mêmes limitations
- **Portage avec une API standard** : le code source est compilé pour chaque OS, la portabilité joue au niveau du code source (POSIX)
- La portabilité « **binaire** » n'existe quasiment jamais

POSIX normalise la compatibilité du code source entre Unix-like

En pratique : combiner plusieurs approches selon les besoins

API vs ABI

- **API (Application Programming Interface)** : interface au niveau du code source — noms, signatures, types (documentée par les en-têtes et man)
- **ABI (Application Binary Interface)** : interface au niveau binaire — format des exécutable, largeur des types, convention d'appel, organisation de la pile, format des appels système
- Un binaire conforme à un ABI s'exécute sur tout système supportant cet ABI
- Un ABI est défini pour un SE précis sur une architecture donnée (ex. ARMv8)
- L'ABI est l'équivalent machine de l'API

Conséquence : l'ABI ne fournit que peu de portabilité inter-systèmes

Cas pratiques : WSL et Wine

- **WSL (Windows Subsystem for Linux)** : exécute des binaires Linux ELF sous Windows 10+
- Une « instance Linux » (init, bash) tourne dans des processus Pico
- Les services LXCORE/LXSS traduisent les appels système Linux en appels Windows
- `fork()` (Linux) est partiellement simulé sur `CreateProcess()` (Windows)
- **Wine** : implémente l'API Windows au-dessus de Linux pour exécuter des programmes Windows

Deux approches opposées :

WSL traduit au niveau des appels système ; Wine au niveau de l'API



WSL

WINE



Exemples d'appels système Windows et Unix

Ce qui suit illustre différents appels système équivalents pour les systèmes d'exploitation Windows et UNIX.

	Windows	Unix
Process control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File management	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device management	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()

Section 8 : Conception et implémentation du SE

Objectifs de conception → mécanismes vs politiques → langage d'implémentation

Objectifs de conception : utilisateurs vs système

- **Objectifs utilisateurs** : pratique à utiliser, facile à apprendre, fiable, sûr, rapide
- **Objectifs système** : facile à concevoir, implémenter et maintenir ; flexible, fiable, sans erreur, efficace
- Ces exigences sont vagues et souvent contradictoires
- Il n'y a pas de solution unique : chaque environnement impose ses priorités
- VxWorks (temps réel) vs Windows Server (grand système multi-accès) : des exigences opposées

Spécifier et concevoir un SE est une tâche hautement créative

Mécanismes vs politiques

- **Mécanisme** : comment accomplir une tâche. **Politique** : ce qui doit être fait
- **Exemple** : le timer est un mécanisme de protection CPU ; le réglage de sa durée est une politique
- Séparer les deux garantit la **flexibilité** : changer de politique ne demande pas de toucher au mécanisme
- **Micro-noyaux** : des blocs de base presque « sans politique », étendus par des modules ou programmes
- À l'opposé, **Windows** a figé mécanismes et politiques pour imposer une apparence uniforme. Dans **Linux** par exemple, le mécanisme d'ordonnancement (CFS) supporte la politique ; chacun peut le modifier.

Langage d'implémentation

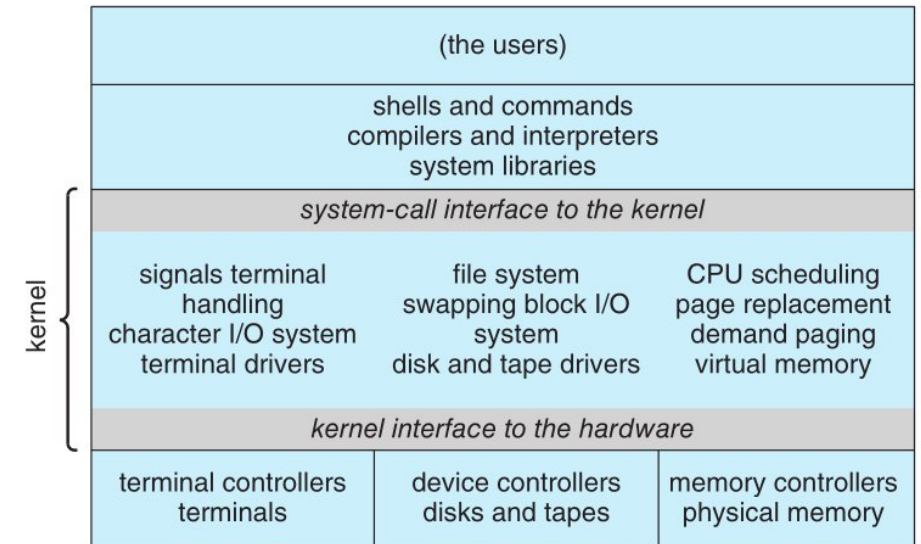
- Les premiers SE étaient écrits en assembleur
- **Aujourd'hui** : essentiellement du C/C++, avec de petites portions en assembleur (les plus bas niveaux (comme la gestion des interruptions))
- Avantages des langages de haut niveau : développement plus rapide, code plus compact et lisible, plus facile à déboguer
- Portabilité : recompiler suffit pour changer de matériel (x86, ARM, embarqué)
- Le compilateur moderne produit souvent un meilleur code qu'un expert en assembleur
- **Android** : noyau en C, bibliothèques en C/C++, frameworks en Java

Section 9 : Les structures du SE

Monolithique → Linux → en couches → micro-noyau → modules → hybrides → comparaison

La structure monolithique

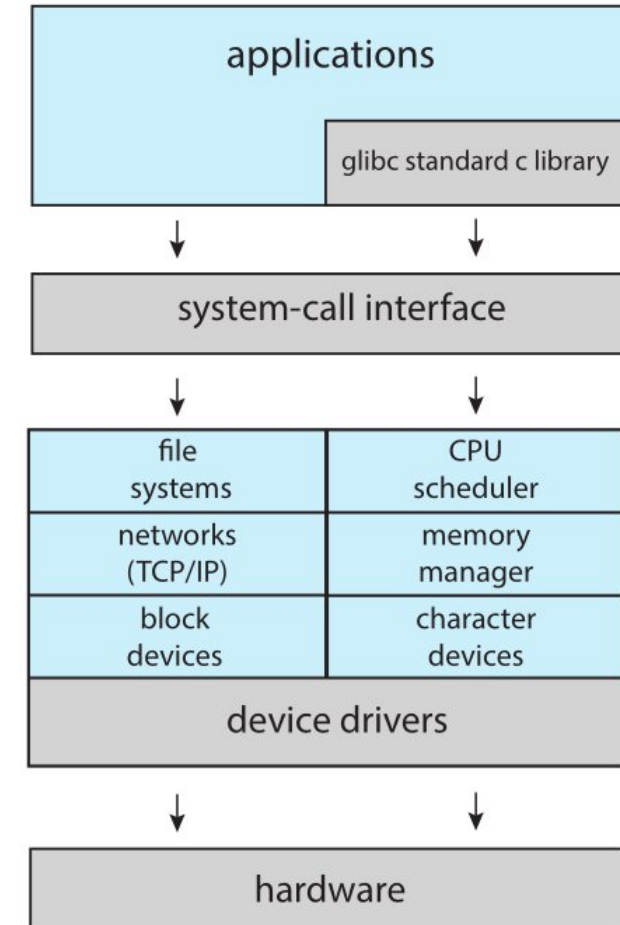
- **Structure monolithique** : toutes les fonctionnalités du noyau dans un seul binaire statique, dans un seul espace d'adressage
- C'est « **l'absence de structure** » : tout est dans le noyau
- **UNIX traditionnel** : le noyau fournit fichiers, ordonnancement CPU, gestion mémoire, pilotes,...
- Tout ce qui est au-dessus de l'interface d'appels système et au-dessous du matériel = noyau
- **Avantage** : performances élevées — peu de surcoût, communication interne rapide
- **Inconvénients** : difficile à implémenter, à étendre ; un bug peut faire tomber tout le système



Traditional UNIX system structure.

Linux : un noyau monolithique modulaire

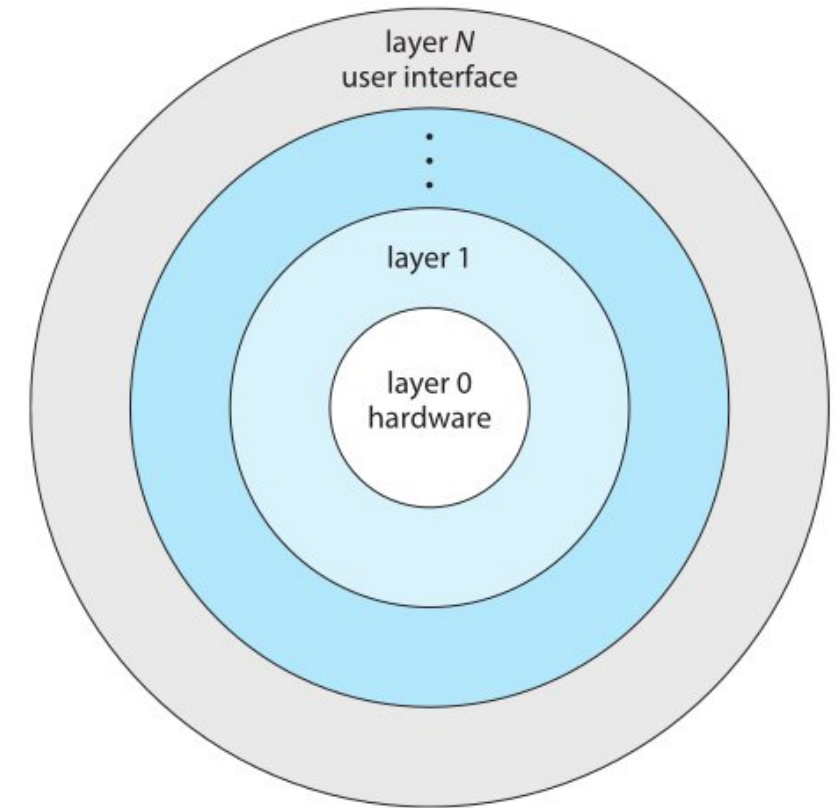
- Applications → glibc → interface d'appels système
- Dans le noyau : systèmes de fichiers, ordonnanceur CPU, réseau TCP/IP, gestion mémoire
- Périphériques : blocs et caractères, via des pilotes
- Linux est monolithique (tout en mode noyau, un seul espace d'adressage)
- Mais modulaire : des modules peuvent être chargés et déchargés à chaud
- **Résultat** : la performance du monolithique, la flexibilité des modules



Linux system structure.

L'approche en couches

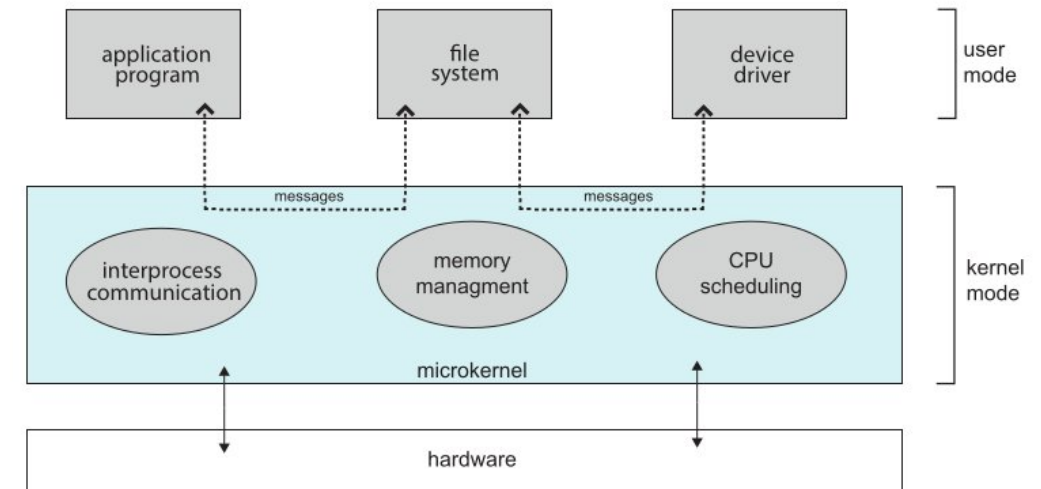
- **Approche en couches** : le SE est découpé en niveaux (layer 0 = matériel, layer N = interface utilisateur)
- Chaque couche utilise uniquement les services des couches inférieures
- **Avantages** : construction et débogage simples — on débogue couche par couche, de bas en haut
- Chaque couche masque ses données et opérations aux couches supérieures
- **Inconvénients** : définition difficile des couches, performance dégradée (traverser toutes les couches)
- Peu de SE sont purement en couches ; on retrouve l'idée dans TCP/IP et les applications web



A layered operating system.

Le micro-noyau

- **Micro-noyau** : ne conserve en mode noyau que l'essentiel — gestion minimale des processus et de la mémoire, communication
- Tous les autres services (fichiers, pilotes) deviennent des programmes en espace utilisateur
- Communication par passage de messages à travers le micro-noyau
- Né à Carnegie Mellon avec **Mach** (années 1980) ; aussi **QNX** (temps réel)



Architecture of a typical microkernel.

Avantages : extension facile, meilleure sécurité et fiabilité — un service qui plante ne fait pas tomber le système

Inconvénient majeur : surcoût du passage de messages → performances en retrait

Les modules chargeables (LKM)

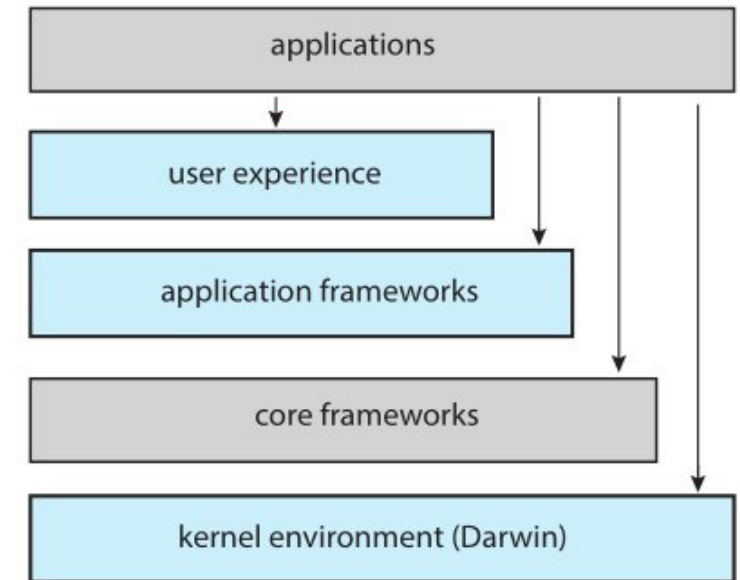
- **Modules chargeables (LKM)** : le noyau fournit un noyau de services de base ; des modules s'y greffent au démarrage ou à chaud.
- Les modules communiquent directement, sans passage de messages
- **Exemples** : pilotes de périphériques (USB), systèmes de fichiers
- **Insertion** : au boot ou pendant l'exécution (brancher un périphérique USB)
- **Avantages** : pas de recompilation du noyau à chaque ajout ; souplesse du micro-noyau sans le surcoût des messages
- **C'est le design de Linux, macOS, Solaris et Windows**

Systèmes hybrides

- En pratique, très peu de SE adoptent une structure unique et strictement définie : **la majorité sont hybrides**
- Un hybride combine plusieurs structures pour concilier performance, sécurité et facilité d'usage
- **Linux** : monolithique (un seul espace d'adressage, donc performant) mais aussi modulaire (fonctionnalités ajoutées dynamiquement)
- **Windows** : majoritairement monolithique, mais conserve des traits micro-noyau — des sous-systèmes en mode utilisateur (les « personnalités ») et des modules noyau chargeables
- Trois études de cas, détaillées aux slides suivants : **macOS & iOS, Darwin, Android**

Systèmes hybrides : macOS et iOS

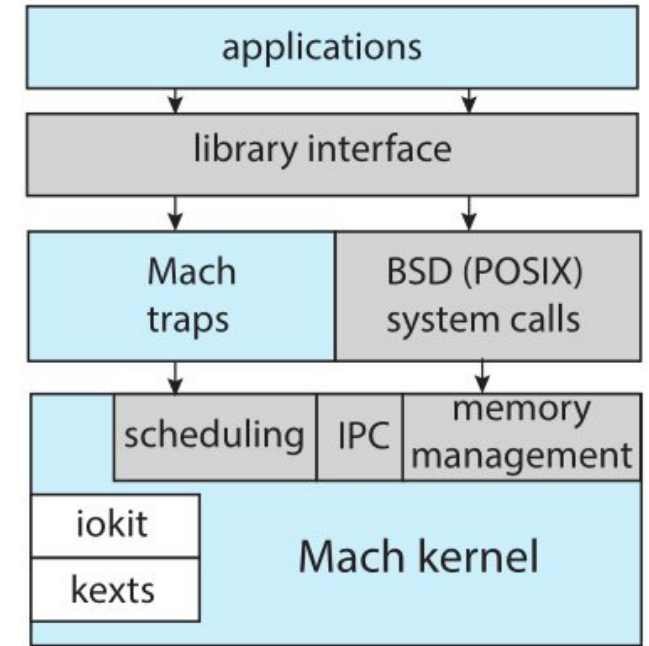
- Apple présente **macOS** (bureau/portable) et **iOS** (iPhone/iPad)
- **Couche expérience utilisateur** : **Aqua** (souris/pavé) pour macOS, **Springboard** (tactile) pour iOS
- **Couche frameworks applicatifs** : **Cocoa** (macOS) vs **Cocoa Touch** (iOS, fonctions tactiles) — API Objective-C/Swift
- **Couche frameworks cœur** : graphisme et multimédia (Quicktime, OpenGL)
- **En bas, l'environnement noyau** : Darwin (Mach + BSD)
- Une application peut court-circuiter les couches et parler directement au noyau (ex. un programme C POSIX sans interface)
- **Différences** : macOS compilé pour Intel, iOS pour ARM ; noyau iOS adapté (énergie, mémoire) ; iOS plus verrouillé (accès POSIX/BSD restreint)



Architecture of Apple's macOS and iOS operating systems.

Darwin : un noyau hybride (Mach + BSD)

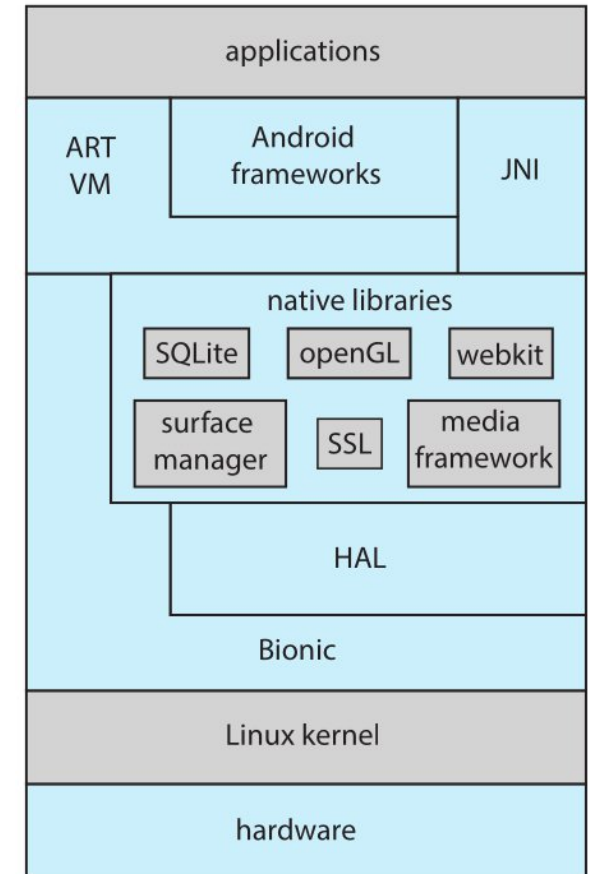
- **Darwin** = structure en couches combinant le micro-noyau **Mach** et le noyau **BSD UNIX**
- Deux interfaces d'appels système (au lieu d'une seule, comme Unix/Linux) :
 - **les traps Mach** ;
 - **les appels système BSD (POSIX)**
- Mach fournit les services fondamentaux : gestion mémoire, ordonnancement CPU, IPC (passage de messages, RPC)
- Abstractions du noyau : tâches (processus Mach), threads, objets mémoire, ports
- **Exemple** : `fork()` (BSD/POSIX) → Mach crée une tâche en interne
- I/O Kit pour les pilotes + extensions noyau chargeables (« kexts »)
- Mach, BSD, I/O Kit et kexts sont fusionnés dans un seul espace d'adressage → le passage de messages ne copie plus rien
- **Darwin est open source ; les frameworks Cocoa d'Apple restent propriétaires**



The structure of Darwin.

Android

- **Android** : conçu par l'Open Handset Alliance (menée par Google), open source, tourne sur une multitude d'appareils
- Pile logicielle en couches (comme iOS) : applications, **ART**, frameworks, bibliothèques natives, HAL, noyau Linux
- Applications écrites en **Java** avec une **API Android** dédiée (pas l'API Java standard)
- **Compilation** : Java → bytecode `.class` → `.dex`, exécuté par **ART** (Android RunTime)
- **ART** fait de la compilation ahead-of-time (AOT) : plus efficace et moins gourmand en énergie
- **JNI** : contourne la machine virtuelle pour accéder au matériel (peu portable)
- **Bibliothèques natives** : WebKit (navigateur), SQLite (base de données), SSL, OpenGL, Surface Manager, Media Framework
- **Bionic** : bibliothèque C standard d'Android, plus légère que glibc, adaptée aux CPU mobiles
- **HAL** : abstrait caméra, GPS, capteurs → une application devient portable sur tous les appareils
- **Noyau Linux modifié** : gestion d'énergie, gestion mémoire, IPC « Binder »



Architecture of Google's Android.

Comparaison des structures du SE

	Monolithique	En couches	Micro-noyau	Modules
Performance	Élevée	Faible	Faible	Élevée
Fiabilité	Faible	Élevée	Élevée	Moyenne
Sécurité	Faible	Élevée	Élevée	Moyenne
Extensibilité	Faible	Moyenne	Élevée	Élevée
Débogage	Difficile	Facile	Facile	Moyen

- **Linux** : monolithique + modules → performance et extensibilité
- **macOS** : micro-noyau + BSD fusionnés → hybride
- Le bon choix dépend des exigences : performance, fiabilité, sécurité, coût de développement

Section 10 : Analyse de performance

strace et ltrace

strace : tracer les appels système

- `strace` : exécute un programme et affiche chaque appel système, ses paramètres et sa valeur de retour
- `strace ls` : voir les syscalls de `ls` (`openat`, `getdents`, `stat`, `write`)
- `strace -c ls` : résumé statistique (nombre d'appels par syscall)
- `strace -e openat ls /tmp` : ne tracer que certains syscalls
- `strace -p PID` : s'attacher à un processus en cours d'exécution
- **Exemple**: `strace -o strace.txt vi fichier.txt` → 941 lignes de trace, on y voit `chmod` appelé deux fois

ltrace : tracer les fonctions de bibliothèque

- **ltrace** : trace les appels aux fonctions de bibliothèque (libc) d'un programme
- **ltrace ls** : chaque appel de libc avec ses paramètres et sa valeur de retour (**strchr**, **setlocale**, **getenv**, **isatty**, ...)
- **ltrace -c ls** : résumé statistique — nombre d'appels, temps passé, fonction
- **ltrace -e printf ./prog** : ne tracer que certaines fonctions
- **Complémentaire de strace** : **ltrace** voit la couche libc, **strace** voit les appels système
- **Trois niveaux** : votre code → fonctions de libc (**ltrace**) → syscalls (**strace**)

Quiz : services et appels systèmes



<https://forms.cloud.microsoft/r/KMDzS8CGtu>